

第 14 讲强化学习入门：从策略梯度到 PPO，让宇树 G1 自己学会动

目录

1 从模仿到试错	3
1.1 回顾第 8 讲：策略是从观测到动作的映射	3
1.2 示教教不了的事	3
1.3 用奖励代替标签	3
1.4 本讲的任务与路线：让 G1 学会走路	4
2 把「学走路」写成强化学习问题	4
2.1 智能体与环境：一个回合怎么跑	4
2.2 状态与动作：G1 看到什么、能动什么	5
2.3 奖励与回报：怎么给「走得好」打分	6
2.4 策略与价值：两个基本量	7
2.5 本节小结	7
3 强化学习方法分类	8
3.1 第一个划分：学不学环境模型	8
3.2 第二个划分：学什么——基于价值的方法与基于策略的方法	8
3.3 Actor-Critic：横跨两类的混合架构	9
3.4 第三个划分：on-policy 与 off-policy——学的策略和采数据的策略是不是同一个	9
3.5 把算法放上地图	10
3.6 本节小结	11
4 REINFORCE：最朴素的策略梯度方法	11
4.1 什么是策略梯度	11
4.2 梯度往哪指：好动作加概率，坏动作减概率	12
4.3 REINFORCE：采一批轨迹、算回报、更新一次	12
4.4 认识三个外部依赖：mujoco、GVHMR 与 GMR	13
4.5 代码走读：v1 版训练循环	14
4.6 实验：G1 用 REINFORCE 学走路	16

4.7 毛病在哪：回报噪声淹没了学习信号	17
4.8 本节小结	17
5 A2C：请一位评分员来降方差	17
5.1 基线的直觉：跟「平均水平」比，不是跟零比	17
5.2 价值函数与优势：V、Q、 $A = Q - V$	18
5.3 从基线到 critic：Actor-Critic 与 GAE	19
5.4 代码走读：v1 到 v2 改了些什么	20
5.5 实验：同一个任务，A2C 稳了多少	20
5.6 毛病在哪：更新步子没人管	21
5.7 本节小结	22
6 PPO：给更新步子加护栏	22
6.1 信任域思想：TRPO 一句话，PPO 是它的好实现	22
6.2 概率比值与 clip 目标	22
6.3 数据复用与 KL 自适应	23
6.4 代码走读：v2 到 v3 改了些什么	24
6.5 实验：三个算法同台，行走任务收口	25
6.6 实验加码：动作跟随，任务越难差距越大	28
6.7 本节小结	30
7 本讲小结	30
7.1 一条演进链：每一步为什么发生	30
7.2 回到分类地图：GRPO 与 off-policy 在等下两讲	31
参考资料	32

从第 8 讲开始，我们做的事情其实只有一件：训练一个**策略**，再把它部署到闭环里。第 8 讲立起了这个概念——策略就是从观测到动作的映射；第 9 到 12 讲围着它转了一整圈：采数据、用 ACT 和 VLA 学它、微调它；第 13 讲看了它的前沿形态。这一整段有个共同前提：**训练信号来自人的示教**——每一步都有人告诉网络“该做什么”。

这一讲把这个前提抽掉。有些事情示教教不了：没人能遥操作出人形机器人 29 个关节的行走节奏，示教再多也超不过示范者本人，走到示教没覆盖的状态更是无人可问。这一讲换一种监督信号：不再给“标准答案”，只给一个**能打分的奖励**，让机器人自己**试错**。这就是强化学习（Reinforcement Learning, RL）。

强化学习的概念和算法出了名的多而杂，这一讲不求全，只把入门必须的一条主线走透：先把“学走路”这个任务翻译成强化学习的语言，再建立一张**不会混淆的方法分类地图**，然后沿着地图上“基于策略”这一支，从最朴素的 REINFORCE 出发，每讲完一个算法就在**同一个任务**上跑给你看——看到它的毛病，再引出下一个算法——一路升级到工业级的 PPO。最后把同一套 PPO 搬去做更难的动作跟随，你会看到：任务越难，每一块机制的价值越明显。

1 从模仿到试错

1.1 回顾第 8 讲：策略是从观测到动作的映射

第 8 讲给策略下过定义： π_θ 是一个参数为 θ 的网络，输入观测 o_t ，输出动作 a_t ；部署时它在闭环里反复“看一眼、动一下”。之后几讲无论模型怎么变——ACT、 π_0 、VLA——都没有离开这个框架，变的只是网络结构和训练数据。

本讲的主角**还是同一个** π_θ ，甚至连“闭环交互”这个舞台都没换。真正换掉的只有一样东西：训练信号。模仿学习的每条训练样本都带着“这一步该做什么”的动作标签；强化学习没有标签，只有环境对结果的打分。

备注：符号上有一处小衔接。强化学习教材习惯把策略的输入写成状态 s_t ，第 8 讲写的是观测 o_t 。区别在于“看没看全”：状态是环境的完整信息（**完全可观测**），观测是策略实际看到的部分（真机常常只做到**部分可观测**）。本讲在仿真里把该看的都给了策略，两者暂不区分、统一写 s_t ；真机上“看不全”的麻烦见 2.2 节的备注。

1.2 示教教不了的事

模仿学习的天花板，第 8 讲和第 10 讲都碰到过。集中起来是三件事：

- **有些任务根本采不到合用的示教**。桌面操作可以用主从臂示教（第 9 讲），但人形行走要 29 个关节以每秒几十次的频率协调发力，很难直接采到**足够覆盖、足够稳定、又能当低层控制标签**的示教数据——主从遥操作这类装置尤其够不着（动捕、动作重定向能提供某种参考动作，但那是另一种监督，6.6 节就会这么用）。本讲的任务——让 G1 学走路——恰好就是这种情形。
- **缺少直接优化任务回报的机制**。纯行为克隆的学习目标是“贴近示教动作”，受示教分布与标签质量约束；监督信号里没有“哪种做法回报更高”这个概念。（要注意这不等于“模型上限就是示范者水平”：聚合多个示范者、去噪、泛化，都可能让策略在某些指标上超过单个示范者。真正的限制是数据支持范围、优化目标和评测分布。）
- **错了不会救**。第 8 讲讲过闭环里误差会滚雪球：一旦走到示教数据没覆盖的状态，模型两眼一抹黑，因为那里没有标签。

三件事的根源是同一个：模仿学习的监督信号回答的是“**这一步该做什么**”。而上面三个场景需要的是另一种信号——“**这一步做得好不好**”。

1.3 用奖励代替标签

“做得好不好”用一个标量就能表达：**奖励**。环境每收到一个动作，除了转移到新状态，还吐回一个分数。于是第 8 讲的部署闭环“观测 → 动作 → 环境 → 新观测”上多出一条回路：**每一步都带回一个奖励**。机器人在闭环里反复尝试，把“怎么动能拿高分”自己摸出来。

这个改动顺带改变了 rollout 的身份。第 8 讲里 rollout 是**验证工具**——loss 再低也要拉到闭环里跑过才算数；本讲里 rollout 变成了**训练数据本身**：机器人自己跑出来的轨迹、连同每一步的得分，就是训练集。数据不再是事先采好的静态数据集，而是边训练边生成、随着策略变化而变化——这是强化学习与模仿学习在工程形态上最大的不同，也是后面 3.4 节讨论“这些自产数据怎么用”的伏笔。

1.4 本讲的任务与路线：让 G1 学会走路

贯穿全讲的任务只有一个：**宇树 G1 人形机器人，在平地上按速度指令行走**——每个回合随机给一个目标速度（前进/侧移/转向），走稳、跟上就得分。选它做入门载体有三个理由：它不需要任何示教数据（正是 1.2 节的第一种情形）；“跟上速度、别摔倒”的分数很好写；仿真里可以大规模并行试错，单卡就能复现。

方法上我们走一条**三级阶梯**，每一级都是一个完整算法，在同一个任务上跑出来对照：

- **v1 REINFORCE**：最朴素的策略梯度，能学，但抖、慢、会摔；
- **v2 A2C**：请一个“评分员”（critic）降低学习信号的噪声，更稳一点，但会突然崩；
- **v3 PPO**：给每次更新加上护栏，又稳又省数据——工业级配方。

为了让对照干净，三个版本刻意做到**只有算法不同**：同一个环境、同一个网络结构（`model.py` 里的 `ActorCritic`）、同一份训练预算（4096 个并行环境 × 每轮 24 步 × 3000 迭代）；评测口径也完全一致——各自的最终权重、确定性动作、16 个环境 × 400 步统一打分。三份训练脚本都用标准的 PyTorch Lightning 结构组织：一个在线采样的 `Dataset`、一个 `LightningDataModule`、一个负责算 loss 的 `LightningModule`，入口统一是 `trainer.fit(model, data)`；要调的超参就近写成变量，没有命令行参数层。**版本之间的差异（diff）就是这一讲的知识点**——读懂三个文件的区别，就读懂了从 REINFORCE 到 PPO 的演进。

讲完 PPO，6.6 节还会把同一套代码原样搬到一个难得多的任务——让 G1 逐帧贴住一段真实武术动作——看看任务变难时，三级阶梯的差距会被放大成什么样。

2 把「学走路」写成强化学习问题

强化学习有一套自己的词汇。这一节不空讲定义，每引入一个词，就立刻落到 G1 行走任务上——把“学走路”完整翻译成强化学习的语言，后面所有算法都在这套语言上工作。

2.1 智能体与环境：一个回合怎么跑

强化学习的基本场景只有两个角色：一个**智能体（agent）**在一个**环境（environment）**里行动。智能体每一步看到环境的状态，做一个动作；环境收到动作后，给出一个奖励、并转移到下一个状态。

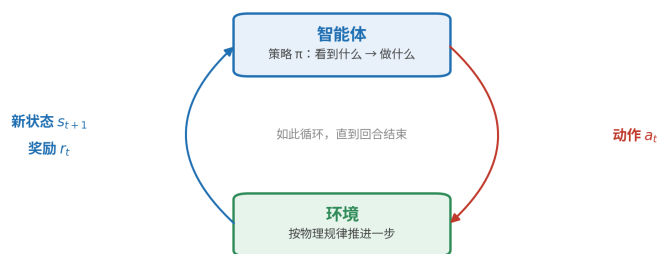


图 1: 智能体与环境的交互闭环: 智能体在状态 s_t 下输出动作 a_t , 环境返回新的状态 s_{t+1} 和奖励 r_t , 如此循环

把一步交互写成符号, 就是:

$$s_t \xrightarrow{a_t} r_{t+1}, s_{t+1}$$

这个抽象框架放到 G1 行走上, 每个角色的具体所指是:

概念	含义	在 G1 行走里的具体所指
智能体 agent	做决策的主体	控制 G1 的策略网络
环境 environment	智能体交互的世界	仿真器里的物理引擎
状态 state	智能体看到的信息	关节角度、躯干姿态、目标速度指令
动作 action	智能体做出的动作	29 个关节的目标量
奖励 reward	环境返回的反馈	跟上速度 + 保持直立 - 各种惩罚
轨迹 trajectory / 回合 episode	一整段交互	从站立开始, 到摔倒或 20 秒超时

“试错”的量从哪来? 仿真里同时开 **4096 个并行环境**, 4096 台虚拟 G1 同时摔、同时爬起来, 一轮就贡献近十万步经验。这是强化学习在仿真里能成的关键工程条件。

2.2 状态与动作: G1 看到什么、能动什么

智能体“看到什么”叫**观测空间**, “能做什么”叫**动作空间**。对入门来说, 动作空间的区分尤其要分清:



图 2: 动作空间分两类：游戏手柄按键是离散动作（有限个选项），自动驾驶的方向盘转角是连续动作（一个实数区间）

- **离散动作空间**：动作是有限个选项里挑一个，比如游戏手柄的几个按键、语言模型从词表里选下一个 token。
- **连续动作空间**：动作是一个或一组实数，比如方向盘转角、机械臂每个关节的目标量。

G1 的状态是关节角度、躯干姿态加上目标速度指令；动作是 **29 个关节的目标量，每个都是连续实数**——高维连续动作空间。记住这个事实，3.2 节讲算法分类时它会直接决定我们选哪一支。

备注：宇树 G1 有多种关节配置——基础版约 23 个自由度，带腰部、灵巧手的版本可到四十多个。本讲实验用的是 mjlab 里的 29 自由度版本 (g1_mocap_29dof)，所以全程按 29 个关节算；换配置只改这个数字，算法一行不用动。

备注：真实机器人还有一层麻烦——它通常不是“完全可观测”的：相机只看到局部画面，物体可能被遮挡，执行还带误差。这类“看不全”的问题在学术上叫部分可观测（POMDP）。本讲在仿真里把状态给得比较全（1.1 节说的 o_t 与 s_t 暂不区分），但要知道真机会更难。

2.3 奖励与回报：怎么给「走得好」打分

奖励 (reward) 是某一步的即时反馈，一个标量。G1 这一帧跟上了目标速度、躯干直立，给正分；动作过猛、关节顶到限位、脚底打滑，扣分。**怎么设计奖励，是强化学习里最关键也最难的一环**——分数写歪了，机器人就会学出“钻空子”的动作；机器人和大模型任务里奖励还常常很稀疏，很久才有一次明确反馈。行走任务的好处是**反馈稠密、每步都有**（速度跟踪、姿态稳定、能耗和关节限位惩罚都能就地算出来），比稀疏奖励好写得多；但也别把它想得太轻松——这几项之间的权重怎么配，仍会显著左右最终步态，写“能跑”和写“跑得好”是两回事。

但训练时我们关心的几乎从来不是单步奖励，而是**从现在开始、未来一整段能拿到多少总分**，这叫**回报 (return)**。一个只顾眼前奖励的策略会很短视：为了这一帧站直而猛地刹住，下一帧反而摔了。回报逼着策略为长远负责。把未来的奖励加起来、按时间打折扣：

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

折扣因子 $\gamma \in [0, 1]$ 控制“看多远”：越接近 1 越重视长期，越接近 0 越只顾眼前。本讲取 $\gamma = 0.99$ ，把它换算成“看多远”有三个好记的数：**有效时间尺度约 100 步**（100 步之后的权重已衰减到 $0.99^{100} \approx 0.37$ ），**半衰期约 69 步**（ $0.99^{69} \approx 0.5$ ）。要换成秒，必须除以控制频率——本实验 G1 环境按 50 Hz 出动作，100 步约 2 秒，而一个 episode 长 20 秒（约 1000 步）。所以 $\gamma = 0.99$ 的视野只覆盖一个 episode 的前十分之一左右，**不是“几乎一视同仁地看完整段”**。同一个 γ 换个控制频率，含义就变了，这是很容易踩的坑。一句话：**奖励是每一步的分，回报是一整段的分；强化学习最大化的是回报。**

2.4 策略与价值：两个基本量

策略 (policy) 沿用第 8 讲的定义，写成概率形式是 $\pi_{\theta}(a | s)$ ——参数为 θ 的网络，在状态 s 下给出动作 a 的概率。

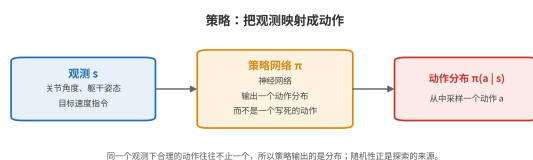


图 3: 策略就是智能体的大脑：输入状态，输出动作（现代 RL 一般输出一个动作分布，再从中采样）

和第 8 讲不同的一点是：本讲用**随机策略**。G1 的策略网络输出每个关节动作的高斯分布（均值 + 标准差），训练时从分布里**采样**去试探，部署时直接取均值。随机性是“探索”的来源——不试新动作，就永远不知道有没有更好的走法；这也是试错学习必须付的入场费。

除了策略，强化学习还有第二个基本量：**价值 (value)**——给“一个状态本身有多好”打分的评分员。现在只需要记住有这么个概念：策略负责“怎么做”，价值负责“值多少分”。它的精确定义（ V 、 Q 、优势 A ）等到第 5 节第一个真正需要它的算法出现时再展开。

2.5 本节小结

“学走路”翻译成强化学习语言：G1 的策略网络是智能体，物理仿真是环境，每回合随机给速度指令，跟上加分、摔倒扣分；4096 个并行环境提供试错的量。G1 的动作空间是 29 维**连续**空间。策略 $\pi_{\theta}(a | s)$ 输出高斯分布、采样探索，目标是最大化折扣回报 R_t 而非单步奖励；价值是“状态值多少分”的评分员，第 5 节正式登场。

3 强化学习方法分类

概念备齐，先别急着钻算法。强化学习的算法出了名的多——REINFORCE、A2C、TRPO、PPO、DQN、DDPG、SAC、AlphaZero.....第一次接触很容易被缩写淹没，而且不同教材的分类词还长得很像，一混就乱。这一节把地图画清楚：三个互相独立的划分，每个算法对三个问题各回答一次，就有了自己唯一的坐标。这张地图不但罩住本讲，也罩住后面两讲。

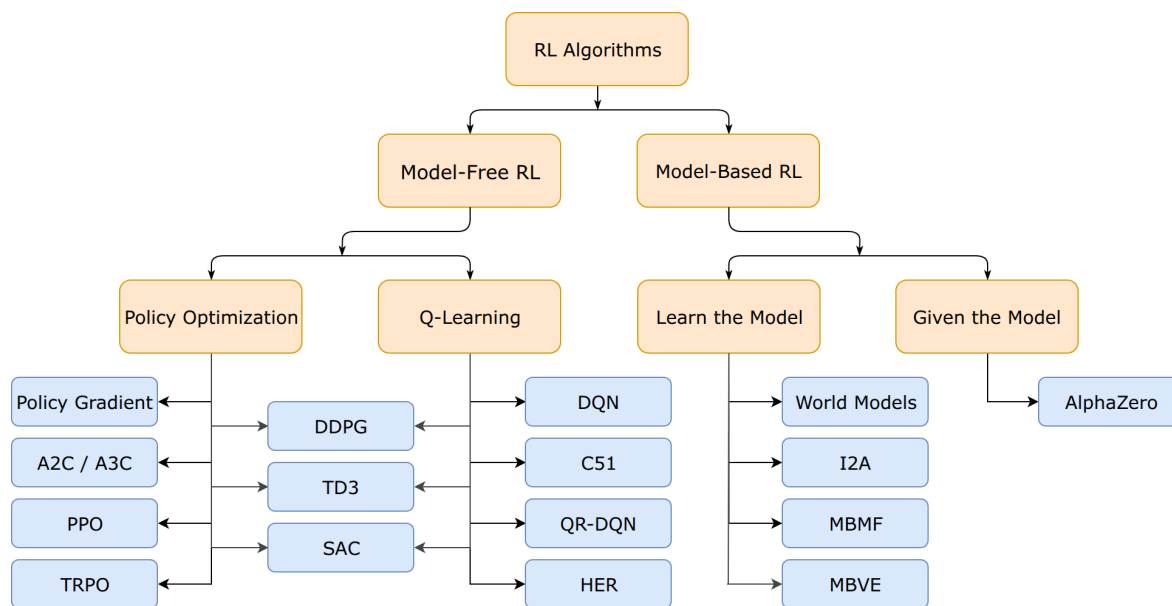


图 4: Spinning Up 的经典算法分类树：第一层按“学不学环境模型”分成 Model-Free 与 Model-Based；Model-Free 内部按“学什么”分成 Policy Optimization（基于策略）与 Q-Learning（基于价值）两族，DDPG/TD3/SAC 画在两族之间

3.1 第一个划分：学不学环境模型

上图的第一刀：**model-based** 方法先学（或已知）一个“环境会怎么变”的模型，再拿它做规划——图右侧的 World Models、AlphaZero 都在这支。**model-free** 方法不显式学环境的转移模型、也不用它做规划，直接拿试错经验去更新策略或价值。机器人任务的环境模型（接触、摩擦、碰撞）极难学准，主流走 **model-free**，即图的左半边——本讲到第 16 讲的所有算法都在这半边，后面不再讨论 model-based。

3.2 第二个划分：学什么——基于价值的方法与基于策略的方法

model-free 内部，按“网络到底在学什么”分成两大类。这是整张地图最重要的一刀：

- **基于价值的方法 (value-based)**：不直接学策略，而是学**动作价值** $Q(s, a)$ ——“在状态 s 做动作 a ，往后平均能拿多少分”。学好之后，策略是**导出**的：每一步挑 Q 值最大的动作。这一族的更新

工具是贝尔曼方程/时序差分（第 16 讲展开），代表算法是 Q-learning 和它的深度版 DQN——就是分类树 “Q-Learning” 那一列。

- **基于策略的方法 (policy-based)**: 直接参数化策略 π_θ 、直接优化它。其中沿”期望回报对 θ 的梯度”做梯度上升的子类，叫**策略梯度方法 (policy gradient methods)**——这是基于策略一族在深度强化学习里的绝对主流（不用梯度的爬山法、进化策略也属于基于策略，但本讲不涉及）。分类树里这支标着 “Policy Optimization”（策略优化），和 “policy-based” 是同一支的两个名字；它下面挂的 Policy Gradient、A2C/A3C、TRPO、PPO，全是策略梯度方法。**本讲的三级阶梯全部走这一支。**

两族各自擅长什么，用 2.2 节的动作空间就能对上号：动作**离散**（按键、从词表选 token）时，对有限个动作逐个比 Q 很自然，基于价值的方法如鱼得水；动作**连续**（G1 的 29 个关节）时，没法对无限个动作枚举 Q ，直接输出动作分布的基于策略方法要顺手得多。这就是本讲选基于策略一支的第一个理由。

3.3 Actor-Critic：横跨两类的混合架构

Actor-Critic 说的是一种架构：模型里同时有一个策略网络（actor，出动作）和一个价值网络（critic，打分）。它描述”网络怎么搭”，不描述”优化目标是什么”，所以它**横跨**上面两大类：

- 本讲的 A2C 和 PPO 带 critic，但 critic 只是给策略梯度当参照物（第 5 节会讲清楚），**优化主体仍是策略**——它们是策略梯度方法；
- 分类树上画在两族**中间**的 DDPG、TD3、SAC 也带 actor 和 critic，但重心倒过来：critic 按贝尔曼方程更新、是学习主体，actor 更像是”帮连续动作挑 Q 最大值”的执行器——它们介于两类之间，更靠基于价值一侧。

一句话：看到 “actor-critic” 别急着归类，先问一句”它的 critic 是给谁打工的”。

3.4 第三个划分：on-policy 与 off-policy——学的策略和采数据的策略是不是同一个

第三个划分和”学什么”**完全独立**，它问的是数据：**被改进的策略**（目标策略）和**生成数据的策略**（行为策略）是不是同一个？

- **on-policy**：是同一个。数据只在**当前策略附近的一小段窗口内**管用——策略一更新，旧 rollout 的分布就和新策略对不上，不能像经验池那样长期留着反复复用（6.3 节会看到 PPO 抢在数据被推远之前复用几轮，正是在这个短窗口里做文章）——稳，但费样本。
- **off-policy**：不是同一个。历史数据（甚至别的策略采的数据）都能塞进经验池反复复用——省样本，但训练更容易不稳。

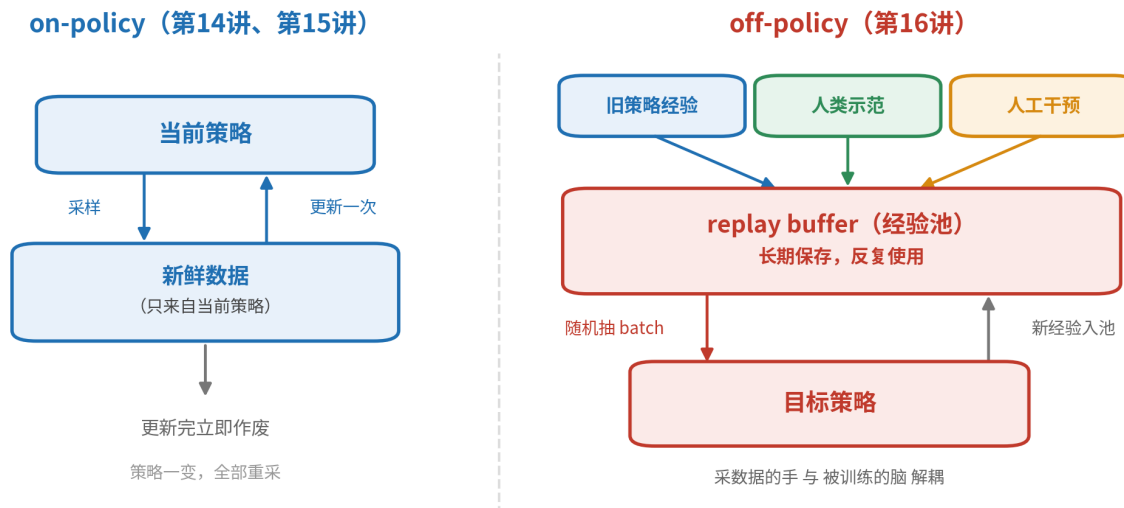


图 5: on-policy 与 off-policy 的数据流对比：左侧数据只来自当前策略、用一次即弃；右侧多来源经验汇入经验池、长期反复使用

两边的差别，落到图上就是**中间那个池子**：左半的数据在“当前策略”和“新鲜数据”之间打转，更新一次就作废；右半的旧策略经验、人类示教、人工干预三种来源全部汇进经验池，目标策略从池里随机抽 batch 训练，新经验只是继续入池。有了这个池子，数据才从消耗品变成资产——这正是第 16 讲上真机时必须换到 off-policy 的原因。

这个划分直接对应一笔经济账：**数据便宜（仿真随便跑）就用 on-policy 图个稳；数据贵（真机一条都不舍得扔）就用 off-policy 把经验池榨干**。本讲在仿真里训练，走 on-policy；第 16 讲上真机，就会换到 off-policy。

备注：**经典的似然比策略梯度** (REINFORCE、A2C、PPO 这一路) 常用 on-policy 估计——因为那个梯度公式里的期望要在**当前策略**的数据分布下估；Q-learning 一族几乎总是 off-policy——因为贝尔曼最优更新跟数据是谁采的无关。但别把它读成“策略梯度必然 on-policy”：**确定性策略梯度 (DDPG、TD3) 和 SAC 都是用策略梯度更新 actor 的 off-policy actor-critic**，它们从回放池里学（本讲后面的分类地图里就有这几个）。所以“基于策略 $\approx$ on-policy、基于价值 $\approx$ off-policy”是**常见对应，不是定义**：两条轴一个说“学什么”、一个说“数据怎么用”，概念上独立。分不清这两条轴，是强化学习分类看着混乱的最大根源。

3.5 把算法放上地图

三个划分立好，把本讲和后面两讲会遇到的算法全部放上去：

算法	学什么	数据	在课程里的位置
REINFORCE	基于策略（策略梯度）	on-policy	第 14 讲 4.3；代码 4.5（v1）
A2C	基于策略（策略梯度 + critic 参照）	on-policy	第 14 讲 5.3；代码 5.4（v2）
TRPO	基于策略（策略梯度 + 硬约束步长）	on-policy	第 14 讲 6.1（一句话带过）
PPO	基于策略（策略梯度 + clip）	on-policy	第 14 讲 6.2；代码 6.4（v3）
GRPO	基于策略（策略梯度，去掉 critic，用组内平均当参照）	on-policy	第 15 讲 1.4
Q-learning / DQN	基于价值	off-policy	第 16 讲 2.4
SAC（及 DDPG/TD3）	两族之间的 actor-critic	off-policy	第 16 讲 5.2

看这张表要抓两条线：

- **本讲的三级阶梯 REINFORCE → A2C → PPO 是同一支内的演进**，不是三个门类：都是 model-free、基于策略（策略梯度）、on-policy，坐标完全相同，差别只在”怎么把策略梯度算得更稳”——这正是第 4 节到第 6 节一章一块要补的机制。演进链再往后一站就是第 15 讲的 GRPO：它保留 PPO 的 clip、去掉 critic，是这条链在大模型后训练场景下的延伸。
- **第 16 讲换的不是”更好的算法”，而是另一格坐标**：真机数据贵，必须 off-policy，于是从基于策略一支跳到 SAC 那一格。到时候你会发现本讲的概念一个都不浪费。

3.6 本节小结

强化学习方法分类只需要三个独立的问题：学不学环境模型（model-free / model-based）、学什么（基于价值 / 基于策略，策略梯度是基于策略的主流子类）、学的策略和采数据的策略是否同一个（on-policy / off-policy）。Actor-Critic 是横跨两类的**架构词**。“基于策略 ≈ on-policy”只是常态对应、不是定义。本讲路线：model-free + 基于策略（策略梯度）+ on-policy，沿 REINFORCE → A2C → PPO 走到底；GRPO（第 15 讲）和 SAC（第 16 讲）都能在这张地图上找到自己的格子。

4 REINFORCE：最朴素的策略梯度方法

4.1 什么是策略梯度

基于策略的方法要做的事一句话：调策略参数 θ ，让期望回报变大。把”期望回报”记作目标函数 $J(\theta)$ ，最直接的做法就是对它做梯度上升。**策略梯度定理**给出了这个梯度可以怎么从采样数据里估出来（结

论直接用，不推导)：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot R_t \right]$$

符号	含义	说明
$J(\theta)$	目标函数	当前策略的期望回报，越大越好
$\log \pi_{\theta}(a_t s_t)$	对数概率	策略当时有多倾向于选这个动作（取对数便于求梯度）
$\nabla_{\theta} \log \pi_{\theta}$	对数概率的梯度	指向“让这个动作概率变大”的参数方向
R_t	从 t 起的折扣回报	2.3 节定义的 reward-to-go，当这一步的“权重”

沿这个梯度做上升的算法，统称**策略梯度方法**——就是 3.2 节点过名、本讲从头走到尾的那条子类链。

4.2 梯度往哪指：好动作加概率，坏动作减概率

上面的公式读起来其实非常直白。 $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 是“让动作 a_t 概率变大”的方向；给它乘上权重 R_t ：

- R_t 大（这一步之后拿了高回报）：往“更常做这个动作”的方向推得狠一点；
- R_t 小甚至为负：反着推，让它以后少出现。

好动作提高概率，坏动作降低概率——按事后的回报大小定推拉的力度。

权重用的是 **reward-to-go** (t 之后的回报) 而不是整条轨迹的总回报，道理也直白：评价 t 时刻的动作，只该算它之后的账；它之前已经发生的奖励与这个动作无关，混进来只添噪声。

落成损失函数，就是在前面加个负号交给梯度下降。下面这一行等会儿会原样出现在 v1 的代码里：

```
policy_loss = -(log_probs * batch["advantages"]).mean()
```

4.3 REINFORCE: 采一批轨迹、算回报、更新一次

把上面的公式变成可执行的循环，就是 REINFORCE，教材上也叫**蒙特卡洛策略梯度**——“蒙特卡洛”指权重 R_t 是用**实际跑出来的回报**直接估的，不借助任何辅助网络：

1. 用当前策略在环境里采一批轨迹；
2. 对每一步算 reward-to-go R_t ；
3. 按 $-\log \pi_{\theta}(a_t | s_t) \cdot R_t$ 更新一次策略；
4. **丢掉这批数据** (on-policy: 策略变了，旧数据即过期)，回到第 1 步。

这是所有策略梯度算法的公共骨架。后面的 A2C 和 PPO 都只是在这四步里“换零件”：A2C 换第 2 步的权重，PPO 改第 3 步的更新方式。

4.4 认识三个外部依赖：mjlab、GVHMR 与 GMR

跑实验之前，先认识几位”幕后工作人员”。下一节代码走读里的 `env.py` 只是薄薄一层包装，真正干活的是几个开源组件。它们是本讲实验的依赖，不是研究对象——每一位用一段话知道它是谁、替我们干了什么，就足够了。

第一位是 **mjlab**，G1 的”练功房”。强化学习靠海量试错吃饭，一个环境一步一步跑太慢，我们的训练一次开 4096 个环境同时采样。mjlab 把 GPU 并行版的 MuJoCo (MuJoCo Warp) 包装成强化学习任务框架：场景、观测、奖励、终止条件都是现成积木，还内置了宇树 G1 的标准任务——本章行走用的平地速度跟随、6.6 节动作跟随用的动作跟踪，都直接取自它。`env.py` 做的只是把它的接口整理成课程顺手的样子。

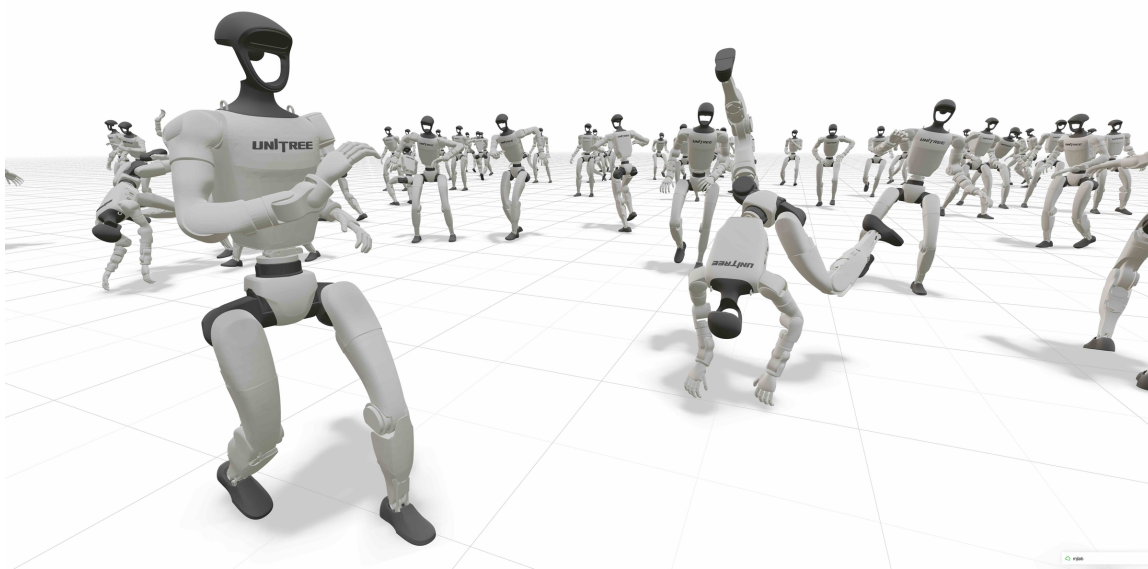


图 6: mjlab 并行仿真中的大批宇树 G1，同屏各自执行行走、翻滚等不同动作，GPU 并行让”海量试错”在一张显卡上成为可能

第二位是 **GVHMR**，负责从视频里”抠”出人的动作。6.6 节要让 G1 逐帧贴住一段真实武术动作，参考动作最方便的来源是一段普通视频。GVHMR (浙江大学团队开源) 输入单目视频，输出世界坐标系下连贯的三维人体动作——人在哪里、朝向哪边、每一帧什么姿势，全部恢复出来，并用 SMPL-X 参数化人体表示 (用一组低维参数描述体型与逐帧姿态，相当于人体动作的通用”语言”)。

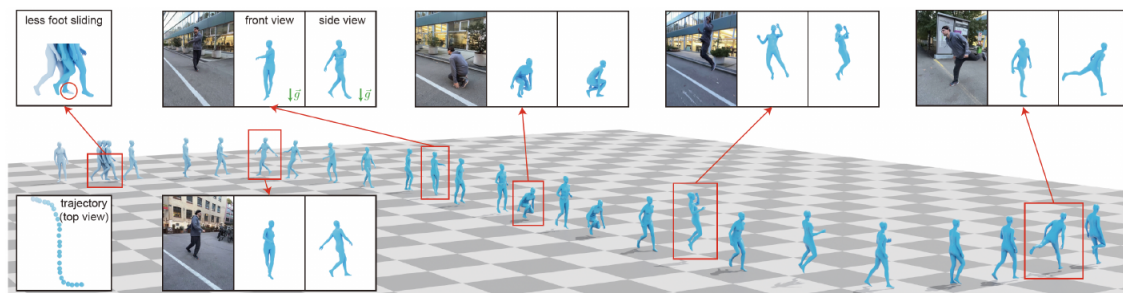


图 7: GVHMR 从街头单目视频恢复世界坐标系下的人体动作：蓝色人体网格为恢复结果，沿真实运动轨迹排开；上方小图给出若干帧的原始画面与恢复姿态的正、侧视对照

第三位是 **GMR** (General Motion Retargeting)，负责把”人的动作”翻译成”G1 的动作”。人和 G1 体型比例不同、关节构成也不同，人的动作不能直接照搬到机器人身上；这道翻译工序的术语叫**重定向** (retargeting)。GMR 先对齐人与机器人的关键部位、按体型做缩放，再用逆运动学逐帧解出 G1 各关节的角度，输出机器人真正能执行的参考轨迹。

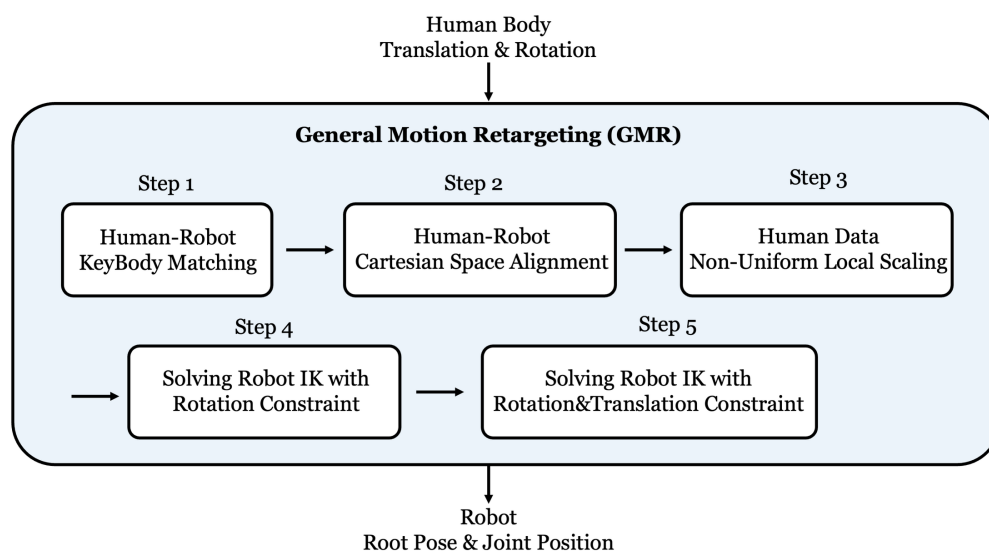


图 8: GMR 的重定向流程：输入人体各部位的平移与旋转，经关键部位匹配、笛卡尔空间对齐、非均匀局部缩放，再两轮求解带约束的机器人逆运动学，输出机器人根姿态与关节位置

从一段视频开始，先用 GVHMR 恢复人体动作，再用 GMR 重定向成 G1 参考动作——这条预处理管线在课程实验仓库里是独立模块，产物是一个参考动作文件。本章的行走实验只用到 mjlab；这份参考动作，等 6.6 节动作跟随登场时才派上用场。

4.5 代码走读：v1 版训练循环

实验代码一共三个文件，三版共用前两个：

文件	职责
<code>env.py</code>	包装 mujoco 的 G1 平地速度任务成 <code>G1WalkEnv</code> ，返回观测/奖励/终止信号
<code>model.py</code>	ActorCritic 网络 (actor + critic 两个 MLP) 与 GAE 工具函数
<code>train_v1_reinforce.py</code>	本节的 REINFORCE 训练脚本

Lightning 结构把训练拆成三块，和 GPT-2 那讲的组织方式完全一样——只是“数据集”从磁盘上的文件变成了**在线采样**：

类	角色
<code>G1WalkRolloutDataset</code>	在线数据集：每轮用当前策略采一段轨迹，产出一个训练 batch
<code>G1WalkData</code>	<code>LightningDataModule</code> ：每个 epoch 重建数据集（保证数据永远新鲜，对应 on-policy）
<code>G1WalkLightningReinforce</code>	<code>LightningModule</code> ：拿到 batch 算 REINFORCE loss

采样端的核心是 reward-to-go 的计算——从后往前累加折扣回报，遇到回合结束就截断：

```
def reward_to_go(reward_steps, done_steps, gamma):
    """ 从后往前累加折扣回报；遇到 done 截断（不跨 episode）。"""
    returns = torch.zeros_like(reward_steps)
    running = torch.zeros(reward_steps.shape[1], device=reward_steps.device)
    for step in reversed(range(reward_steps.shape[0])):
        running = reward_steps[step] + gamma * running * (1.0 - done_steps[step])
        returns[step] = running
    return returns
```

采完 4096 环境 $\times$ 24 步的一段轨迹后，把回报整理成每步的权重：

```
returns = reward_to_go(reward_steps, done_steps, self.gamma)
# 唯一的基线就是整批均值（常数），没有学习的 critic。
advantages = returns - returns.mean()
advantages = advantages / (advantages.std() + 1.0e-8)
```

这里有个值得停一秒的小动作：权重没有直接用 R_t ，而是先**减掉整批均值**再除以整批标准差。这是常见的方差降低与尺度归一化技巧，让数值更平稳。要说准确一点： $b(s)$ 那条“减去与动作无关的基线不改变梯度期望”的性质，要求基线是**固定的、不依赖当前所选动作的**；而这里的批均值恰恰是用同一批的动作和回报算出来的，除以批标准差也会引入有限样本偏差。所以它不是严格无偏，只是在实践里以一点小偏差换取稳定性——好用，但别说成“完全不改变梯度期望”。而“跟平均水平比，而不是跟零比”这个念头，正是下一章整章的种子，先按下不表。

训练端的 `training_step` 就是 4.2 节那行公式，外加一项熵：

```
def training_step(self, batch, batch_idx):
    distribution = self.model.action_distribution(batch["obs"])
    log_probs = distribution.log_prob(batch["actions"]).sum(dim=-1)
    entropy = distribution.entropy().sum(dim=-1)
    policy_loss = -(log_probs * batch["advantages"]).mean()
    entropy_loss = entropy.mean()
    loss = policy_loss - self.entropy_coef * entropy_loss
```

多出来的**熵奖励** (entropy_coef = 0.01) 鼓励动作分布别过早收窄——分布越“摊开”熵越大，等于奖励策略保持探索。这就是 2.4 节说的“随机性是探索的来源”在代码里的落点；三个版本都带这一项，它不是版本间的区别点。

还有一处点破：ActorCritic 网络里其实有 critic，但 v1 的优化器刻意只训 actor——

```
def configure_optimizers(self):
    # 只优化 actor 和动作标准差；critic 不参与训练。
    params = list(self.model.actor.parameters()) + [self.model.std]
    self.optimizer = torch.optim.Adam(params, lr=self.learning_rate)
    return self.optimizer
```

这是为了三版共用同一个网络类、保证对照干净：v1 的 critic 纯粹闲置，REINFORCE 本身完全不需要它。入口照例是 `trainer.fit(model, data)`，其中 `reload_dataloaders_every_n_epochs=1` 让每轮都重新采样——on-policy 的“用完即弃”由 Lightning 的数据加载周期天然实现。

4.6 实验：G1 用 REINFORCE 学走路

按 1.4 节的统一口径评测（最终权重、确定性动作、16 环境 × 400 步）：

指标	v1 REINFORCE
评测平均奖励	0.070
摔倒率	1.3%

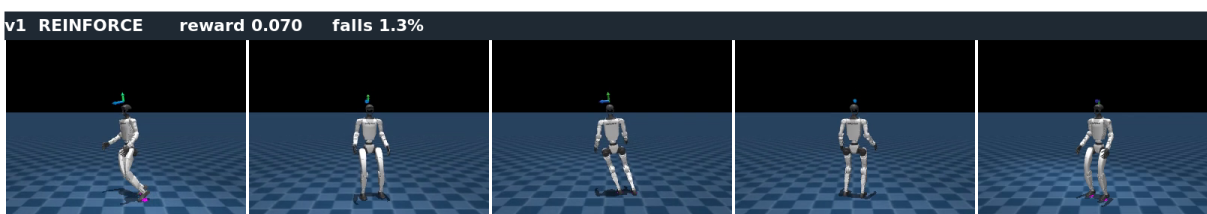


图 9: v1 REINFORCE 训练 3000 迭代后的 rollout 关键帧，从左到右为同一段时间内的连续时刻：G1 能保持站立并小幅挪步，但姿态僵硬、动作幅度小，偶有摔倒

先说成绩：**它真的学会了**。从随机初始化出发——最初的策略只会让 G1 像断线木偶一样瘫倒——训练之后能站住、能顺着指令小幅挪步。这说明”好动作加概率、坏动作减概率”这个信号是实打实起作用的，策略梯度这条路走得通。

再说不足，看图和数字都很明显：动作僵硬、不敢迈步，跟指令跟得很勉强，还有 1.3% 的评测回合以摔倒收场。训练过程中奖励曲线也抖得厉害，涨得慢。能学，但学得**又慢又晃**。

4.7 毛病在哪：回报噪声淹没了学习信号

病根在权重 R_t 上。它是蒙特卡洛估计——**这一步之后实际发生的一切**都被算进这个数里：同一个状态下做同一个好动作，这次后面一路顺风拿了高回报，下次队友（未来的自己）发挥失常摔了，回报就低。于是”这个动作好不好”的信号里，混进了大量**与这个动作无关的运气**。

这就是**高方差**：梯度的方向平均来看是对的，但每一次估计都在真值周围剧烈乱跳。4096 个环境的平均能压掉一部分，但对 29 维连续动作的人形任务远远不够——大部分更新步都在相互抵消，学习自然又慢又晃。

减掉整批均值那个小动作已经暗示了出路：**别拿绝对回报当权重，找个参照物，比出相对好坏**。参照物选得越贴切，运气成分抵消得越干净。什么是最贴切的参照物？下一章的主角。

4.8 本节小结

策略梯度方法沿”期望回报的梯度”直接更新策略，REINFORCE 是它最朴素的实现：用实际回报 (reward-to-go) 当权重，好动作加概率、坏动作减概率，采一批、更一步、丢数据。在 G1 行走上它证明了这条路走得通（能站能挪，0.070 / 摔 1.3%），但蒙特卡洛回报里混着大量与动作无关的运气，**方差大**，学得**又慢又晃**。它的局限指向一个明确的改进方向：给回报找一个像样的参照物。

5 A2C: 请一位评分员来降方差

5.1 基线的直觉：跟「平均水平」比，不是跟零比

评价一个动作，公平的问法不是”它拿了多少分”，而是”它比**这个状态下的平均水平**多拿了多少分”。同样是”后面拿到回报 5”，在一个平均只能拿 1 的烂局面里是神来之笔，在平均能拿 10 的好局面里是拖了后腿。

把这个参照物记作**基线 (baseline)** $b(s)$ ：策略梯度的权重从 R_t 换成 $R_t - b(s_t)$ 。数学上可以证明（这里直接引用结论）：只要 b 不依赖动作，减掉它**不改变梯度的期望**，却能大幅降低方差——运气带来的涨落被参照物同步吸收了。带基线的 REINFORCE 在教材里就叫 **REINFORCE with baseline**。

v1 已经偷偷用了最粗的基线：整批回报的均值，一个常数。但常数对所有状态一视同仁，显然不够贴切——烂局面和好局面的”平均水平”根本不同。**理想的基线应该因状态而异：每个状态自己的平均水平。**这个”每个状态自己的平均水平”，正是 2.4 节预告过的那个量。

5.2 价值函数与优势：V、Q、 $A = Q - V$

现在正式引入价值。先看**状态价值函数** $V^\pi(s)$ ：

$$V^\pi(s) = \mathbb{E}_\pi[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \mid s_t = s]$$

$V^\pi(s)$ 回答”**这个状态本身有多好**”——从 s 出发、之后都按当前策略 π 走，平均能拿到多少回报。它不规定第一步做哪个动作——这正是”这个状态的平均水平”的严格说法，也就是我们要的理想基线。

再看**动作价值函数** $Q^\pi(s, a)$ ，它比 V 多指定了第一步：

$$Q^\pi(s, a) = \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \mid s_t = s, a_t = a]$$

即在状态 s 下**先做动作** a 、之后再按 π 走的平均回报（3.2 节基于价值的方法学的就是它）。两者一减，得到全场最重要的量——**优势函数 (advantage)**：

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

它回答的问题正是 5.1 节那个公平的问法：**这个动作比该状态的平均水平好多少？**

- $A(s, a) > 0$ ：比平均好，**提高**它的概率；
- $A(s, a) < 0$ ：比平均差，**降低**它的概率；
- $A(s, a) \approx 0$ ：和平均差不多，基本不用动。

符号	含义	说明
$b(s)$	基线	只依赖状态、不依赖所选动作的参照物；这类基线减掉后不改变梯度期望
$V^\pi(s)$	状态价值	状态 s 的平均水平——理想的基线
$Q^\pi(s, a)$	动作价值	在 s 先做 a 有多好
$A^\pi(s, a)$	优势	$Q - V$ ，动作比平均好多少——理想的权重

从此策略梯度的权重升级为优势：**估出每个动作的 A ，优势为正就调高概率、为负就调低**。本讲余下的一切、以及第 15 讲的 GRPO，全在围绕”怎么把 A 估得又准又稳”做文章。

5.3 从基线到 critic: Actor-Critic 与 GAE

$V^\pi(s)$ 没有解析解，那就用老办法——再训一个小网络去估它，输入状态、输出一个标量。这个价值网络就是 **critic**（评分员），原来的策略网络对应地叫 **actor**（演员）：演员出动作，评分员打分。

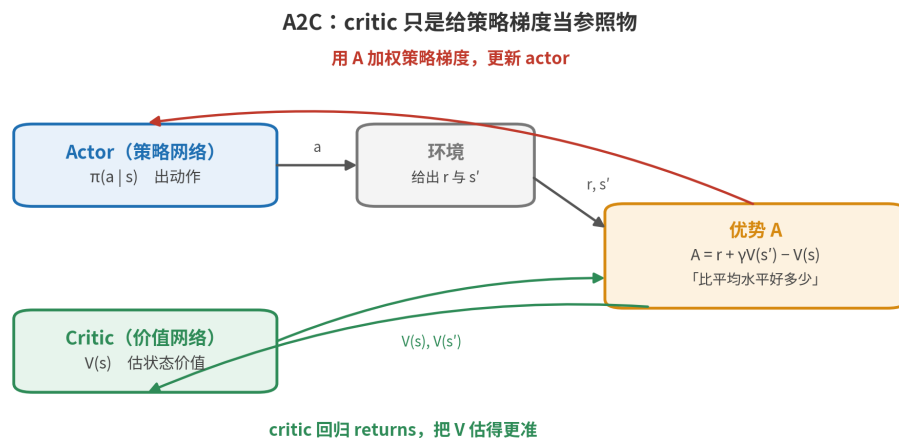


图 10: A2C 的 Actor-Critic 分工：actor（策略网络）出动作，critic（价值网络）估状态价值、帮 actor 算优势 $A = r + \gamma V(s') - V(s)$ ；优化主体仍是策略，critic 只当参照物

这里要把一条容易含糊的界线划清楚——**REINFORCE 和 Actor-Critic 到底差在哪**。文献里的命名并不完全统一：不少资料把“actor + 学出来的 critic 基线”也叫 actor-critic，哪怕 actor 用的是蒙特卡洛回报。本讲采用较窄的定义，以是否 **bootstrap**（自举）为界，即看价值网络的估值参不参与构造回报目标；读别的教材时留意它用的是宽还是窄定义：

- 如果学出来的 V 只当基线——权重用 $R_t - V(s_t)$ ，其中 R_t 仍是实打实跑出来的蒙特卡洛回报——那还是 REINFORCE with baseline，教材明确说这**不算** actor-critic： V 只是个参照物，没有介入回报本身的估计；
- 只有当 V 的估值替代了部分实际回报，比如用时序差分残差 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 来估优势——“这一步实际拿到的，比评分员预期的多多少”——评分员才真正参与了估计本身，这才叫 **actor-critic**。

实践里最常用的优势估计是 **GAE**（广义优势估计）：把一步、两步、多步的时序差分残差按 λ 加权平滑成一个估计。 λ 是方差-偏差的旋钮：靠近 1 更接近蒙特卡洛（准但噪），靠近 0 更依赖评分员（稳但依赖评分员的水平）。本讲取 $\lambda = 0.95$ ，业界常用值。GAE 的公式细节不展开，代码里它就是一个从后往前的递推函数。

把“演员按 GAE 优势做策略梯度、评分员同时回归回报把自己越练越准”组合起来，就是 **A2C**（**Advantage Actor-Critic，优势演员-评分员**）——名字直接点明两个关键词。它还有个多机异步的工程变体 A3C，思路相同。回看 3.3 节的地图定位就清楚了：A2C 的评分员用的正是基于价值一族的时序差分工具，所以说 actor-critic 是“结合两类方法的混合架构”；但评分员在这里是给策略梯度打工的，优化主体仍是策略——A2C 稳稳落在基于策略这一支。

5.4 代码走读：v1 到 v2 改了什么

`train_v2_a2c.py` 与 v1 结构完全相同，diff 集中在两处。第一处在采样端：优势不再是“回报减均值”，而是 critic 估值 + GAE：

```
# 关键变化 (1): 优势不再用常数基线，改用 critic 估值 + GAE (方差大幅下降)
with torch.no_grad():
    next_value = self.model.value(critic_obs)
    advantages, returns = compute_gae(
        reward_steps, done_steps, value_steps, next_value, self.gamma, self.lam)
```

第二处在训练端：评分员自己也要学——多了一项让 V 回归实际回报的 value loss，优化器也从“只训 actor”改成训全部参数：

```
# 关键变化 (2): critic 要回归 returns, 于是 loss 多了一项 value_loss
# 带基线的策略梯度: advantage 已减去 critic 估值; 不做 ratio 裁剪。
policy_loss = -(log_probs * batch["advantages"]).mean()
value_loss = torch.square(values - batch["returns"]).mean()
entropy_loss = entropy.mean()
loss = policy_loss + self.value_loss_coef * value_loss - self.entropy_coef * entropy_loss
```

备注：代码里 critic 吃的是单独的 `critic_obs`——比 actor 的观测多了脚部接触等**特权信息**。这是仿真强化学习的常用技巧（非对称 actor-critic）：评分员只在训练时存在，部署时只带走 actor，所以评分员可以看仿真器里的“天眼”信息，把分打得更准，而不破坏部署的可行性。

5.5 实验：同一个任务，A2C 稳了多少

同一口径评测，先看数字，和 v1 摆在一起：

指标	v1 REINFORCE	v2 A2C
评测平均奖励	0.070	0.043
摔倒率	1.3%	1.3%



图 11: v2 A2C 训练 3000 迭代后的 rollout 关键帧，从左到右为同一段时间内的连续时刻：G1 动作幅度比 v1 明显打开、姿态更舒展，但稳定性没有同步提高，偶有摔倒

结果出乎意料：**评测奖励反而比 v1 低了** (0.043 对 0.070)，摔倒率也没降。这不是 bug，是这一讲最好的教材，值得停下来仔细阅读：

- 先说清一件容易夸大的事：方差被压低是 critic 基线的**理论性质**（5.1 节那个“减掉贴合状态的基线、不改变梯度期望却降方差”的结论），**不是**这张 rollout 对照图或“评测平均奖励”能直接证明的——真要验证方差降没降，得去看训练时的优势/梯度方差曲线。这张图能直接读到的是另一回事：v2 动作幅度更大、姿态更舒展，比 v1 更敢探索大动作。方差与“更敢动”是两码事，别把后者当成前者的证据。
- 但训练过程出现了新的故障模式：奖励涨到一半会**突然崩掉**，好不容易学到的步态一夜回到解放前，再慢慢爬起来，到 3000 迭代预算用完时正好停在一个不高的位置。

按理更干净的学习信号应当帮上忙，为什么反而会崩？一个可能的**解释**是：信号方差降低后，每次更新的推力更一致、有效步长变大——而这套算法里没有任何东西在管“一步最多能迈多大”。这只是一个解释，不是定论。要真正确认“崩掉是因为步子太大”，需要的证据是：优势/梯度方差曲线、新旧策略的 KL、更新范数、学习率扫描，以及多个随机种子的对照——这些量在单次课堂预算的运行里看不到。支持这个解释的间接证据来自 v3：补上步长约束后，同样的优势估计立刻兑现成最高分，方向与“卡在步长”一致。

5.6 毛病在哪：更新步子没人管

策略梯度只给方向，不给安全步长。学习率是固定的，优势偶尔被高估时，一次更新就可能把策略推得离原来很远——高斯分布的均值猛移、标准差猛缩，策略性格突变。

对 on-policy 方法（3.4 节）这尤其致命，因为它有个**恶性循环放大器**：策略崩了 → 下一轮只能用**崩了的策略**采数据 → 数据也是坏的 → 在坏数据上更新，越练越歪，很难爬回来。v1 反而因为信号噪声大、单步推力互相抵消，误打误撞地“步子小”；v2 把噪声压干净，等于给车换了好引擎却没装刹车。

所以下一块要补的机制已经呼之欲出：**限制每次更新的步长**——这正是 PPO 的全部内容。

5.7 本节小结

A2C 在 REINFORCE 上补的是”参照物”：基线思想（跟平均比、不跟零比）→ 理想基线就是状态价值 $V^\pi(s)$ → 用 critic 网络去估它。与 REINFORCE 的分界线（本讲取窄定义）是 bootstrap: V 只当基线仍是 REINFORCE with baseline, V 参与时序差分/GAE 估优势才是 actor-critic。两者用的是**同一条策略梯度公式**，只是权重从实际回报换成了评分员参与估计的优势。理论上 critic 基线会压低梯度方差（实验里我们直接看到的则是动作更舒展、更敢探索，这不等同于方差的直接证据）；本次运行中它没有步长约束、训练会突然崩掉，评测反而不如 v1 (0.043 对 0.070) ——这指向下一块拼图：管住步子。

6 PPO: 给更新步子加护栏

6.1 信任域思想：TRPO 一句话，PPO 是它的好实现

“每次更新别离旧策略太远”这个思想，学名叫**信任域 (trust region)**：只在旧策略周围一小圈范围里更新，才信得过、不会崩。先把这个思想做严格的是 **TRPO** (Trust Region Policy Optimization)：用 KL 散度硬约束新旧策略的距离，再用二阶优化求解——理论漂亮，实现笨重。

PPO (Proximal Policy Optimization, 近端策略优化) 是它广为流传的简化版：“Proximal (近端)”说的就是新策略要待在旧策略附近。它不解带约束的优化问题，只在目标函数上动一个聪明的手脚——裁剪——一阶方法就能跑，效果却不输。稳定、通用、好实现，这让 PPO 成了机器人控制和大模型对齐的默认选择。

6.2 概率比值与 clip 目标

要约束”离旧策略多远”，先得有个量来度量它。PPO 用**新旧策略对同一个动作的概率比值**：

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

$r_t = 1$ 表示新旧策略对这个动作看法一致； > 1 是新策略更喜欢它， < 1 是更嫌弃它。优势为正时我们想把 r_t 往上推，为负时往下压——但 PPO 不允许推拉得太夸张。它把目标函数**裁剪 (clip)** 在 $[1 - \epsilon, 1 + \epsilon]$ 内 (ϵ 取 0.2)：

$$L^{\text{CLIP}} = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

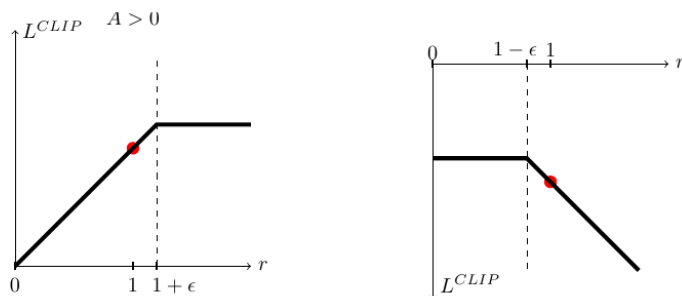


图 12: PPO 的裁剪目标 L^{CLIP} 随概率比值 r 的变化: 左图优势为正 (好动作), 右图优势为负 (坏动作), 红点是 $r = 1$ 的起点。 r 一旦越过裁剪边界, 目标就变平、梯度归零, 更新自动失去动力

这张图分两种情况读:

- **左图 ($A > 0$, 好动作):** 想增大概率、把 r 往右推; 但 r 一旦越过 $1 + \epsilon$, 曲线就**变平**——再推目标也不涨, 梯度归零, 更新自动停手, 好动作的概率不会被推到离谱。
- **右图 ($A < 0$, 坏动作):** 对称地想把 r 往左压; 越过 $1 - \epsilon$ 后曲线同样变平, 不会一脚把坏动作踩到零。

以 $\epsilon = 0.2$ 说人话: 一旦新策略把某个动作的概率推出旧策略的 0.8 到 1.2 倍, 目标函数就不再奖励继续推。要点在于——**clip 夹的是”目标函数的收益”, 不是硬卡住比值 r_t 本身**: 某个样本的 r_t 仍然可能越过这个区间, 只是越界之后再推也赚不到分、梯度归零。“走太远”在目标函数里赚不到钱, 每次更新于是被温柔地拉回旧策略附近。

备注: 所以 PPO-Clip 只是一道**软**信赖域——它是在代理目标上做的软约束, **不保证策略始终待在一个严格的信任域里**。它不像 TRPO 那样用 KL 硬约束新旧策略的距离, 只是让越界的更新”无利可图”; 实际训练中 KL 仍可能超标, r_t 也可能被别的样本带出区间。正因为 clip 不是铁栅栏, 工业级 PPO 才要再配 6.3 节的 KL 自适应学习率兜底, 并持续监控 KL、策略熵和梯度范数。本讲后文说”夹进信任域”, 都是这个软的意思。

6.3 数据复用与 KL 自适应

clip 还带来一个重要副产品: **同一批数据可以多用几遍了**。没有 clip 时数据基本只敢用一遍——多用几遍策略就被推飞; 有 clip 抑制过大的概率比更新, PPO 就能把一段 rollout 切成多个 minibatch、训好几个 epoch, 样本效率大幅提高, 顺手缓解了 on-policy 费样本的老毛病。(这是”通常更稳”, 不是安全保证——KL 仍可能超标, 所以说要监控, 下面就说这件事。)

工业级 PPO 通常还加一个自动化步长管家: **按 KL 散度自适应学习率**——每次更新后量一下新旧策略的 KL 距离, 动得太猛 (KL 超标) 就调小学习率, 动得太小就调大, 步长自动收放。至此, 一轮完整的 PPO 训练是:

1. 用当前策略采一批 rollout;

2. critic + GAE 算每步优势 A_t (第 5 节原样保留);
3. 切成多个 minibatch, 用 L^{CLIP} 反复更新多轮, 同时用 value loss 更新 critic;
4. 按 KL 调学习率, 回到第 1 步。

6.4 代码走读: v2 到 v3 改了什么

`train_v3_ppo.py` 在 v2 上加三块, 正好对应上两节。第一块, policy loss 换成裁剪目标——采样时顺手记下旧策略的 `log_probs`, 训练时算比值:

```
# 关键变化 (1): clip——用概率比值把更新夹在信任域内 (见 6.2 节的图)
ratio = torch.exp(log_probs - batch["log_probs"])
unclipped = ratio * batch["advantages"]
clipped = (torch.clamp(ratio, 1.0 - self.clip_param, 1.0 + self.clip_param)
           * batch["advantages"])
policy_loss = -torch.min(unclipped, clipped).mean()
```

第二块在数据端: `G1WalkRolloutDataset` 不再把整段 rollout 打包成一个 batch 用一遍, 而是切成 minibatch、重复多个 epoch 逐个吐出——Lightning 那边一行不用改, 多轮复用完全由数据集实现:

```
# 关键变化 (2): 一段 rollout 切成多个 minibatch、反复训几轮, 样本效率更高
for _ in range(self.num_learning_epochs):
    indices = torch.randperm(usable_size, device=flat["actions"].device)
    for start in range(0, usable_size, mini_batch_size):
        selected = indices[start : start + mini_batch_size]
        mini_batch = {key: value[selected] for key, value in flat.items()}
```

第三块是步长管家, 挂在 Lightning 的 `on_before_optimizer_step` 钩子上:

```
# 关键变化 (3): 按 KL 自适应调学习率——动得太猛减速, 太小加速
def adapt_learning_rate_to_kl(optimizer, kl, desired_kl, min_lr=1.0e-5, max_lr=1.0e-2):
    current_lr = optimizer.param_groups[0]["lr"]
    kl_value = float(kl.detach()).cpu()
    if kl_value > 2.0 * desired_kl:
        current_lr = max(min_lr, current_lr / 1.5)
    elif 0.0 < kl_value < 0.5 * desired_kl:
        current_lr = min(max_lr, current_lr * 1.5)
    optimizer.param_groups[0]["lr"] = current_lr
    return current_lr
```

此外 v3 对 value loss 也做了对称的裁剪 (新估值别离旧估值太远), 思路与 clip 一脉相承。这套“clip + 多轮复用 + KL 自适应”正是 BeyondMimic 训练人形机器人所用的核心配方——等下我们原样把它搬去做更难的任务。

6.5 实验：三个算法同台，行走任务收口

三个版本到齐，统一口径最后对照一次：

版本	算法	评测平均奖励	摔倒率
v1	REINFORCE	0.070	1.3%
v2	A2C (+critic +GAE)	0.043	1.3%
v3	PPO (+clip + 复用 +KL 自适应)	0.097	0.0%

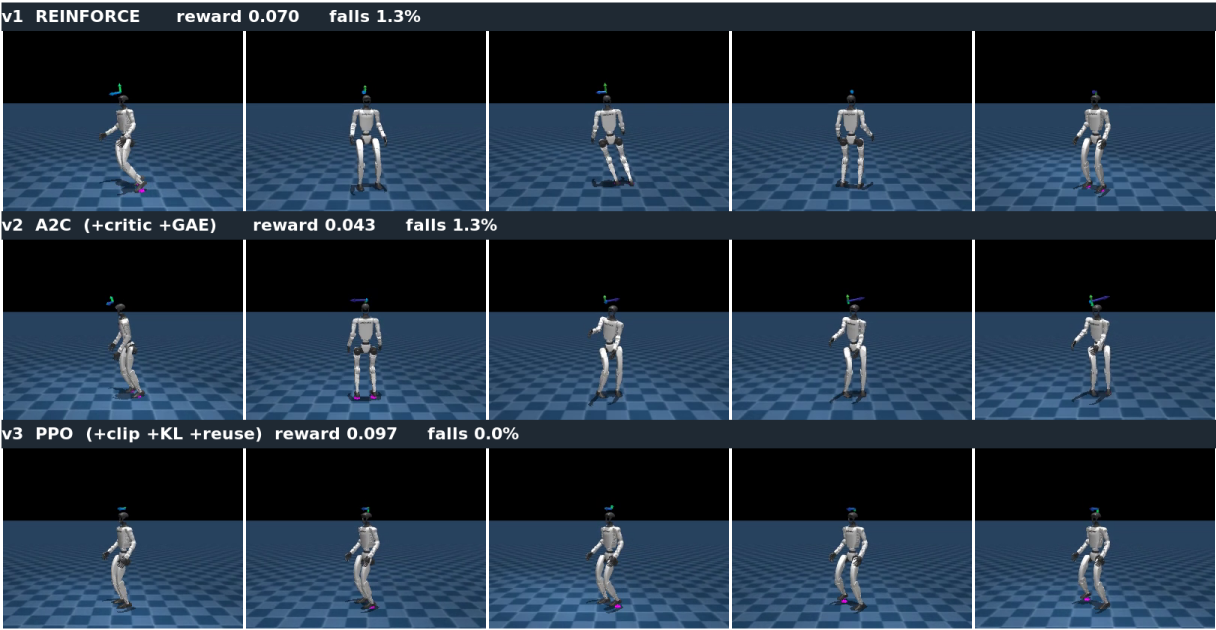


图 13: G1 行走三算法 rollout 对照，每行从左到右是同一段时间内的关键帧，从上到下依次为 v1 RE-INFORCE、v2 A2C、v3 PPO：v3 全程直立稳定、步态最自然，评测中一次未摔

表格和截图给的是终点成绩，把整个训练过程画出来，三者的差别更直白：

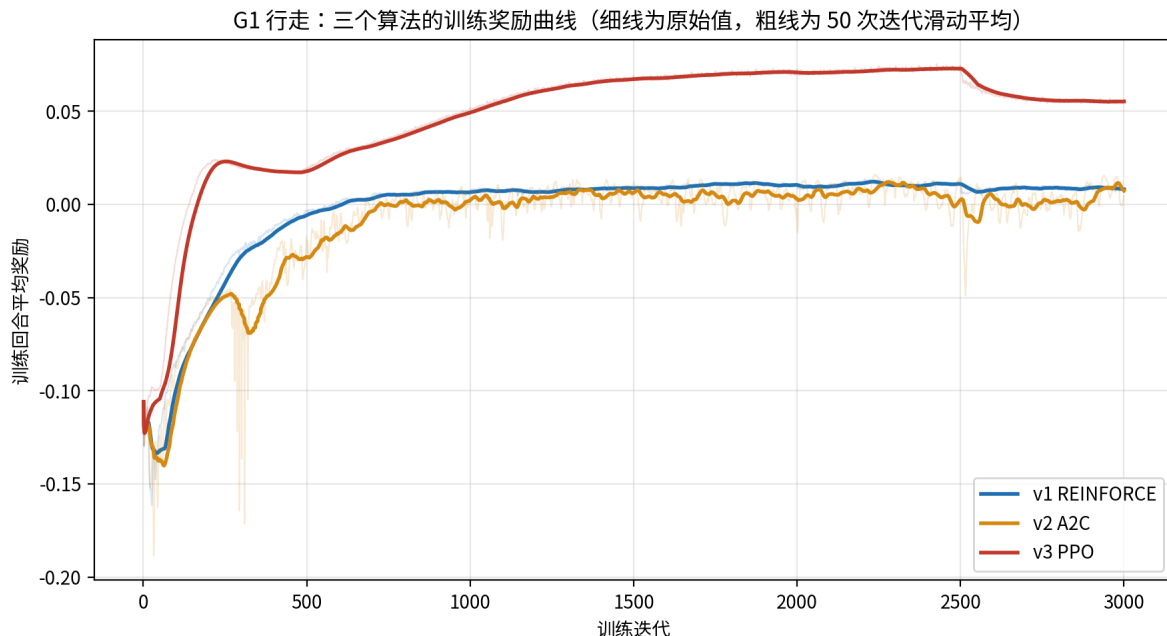


图 14: G1 行走三算法的训练奖励曲线（同为 3000 次迭代，细线为每次迭代的原始值，粗线为 50 次迭代的滑动平均）：v3 PPO 一路爬升且曲线平滑，v1 与 v2 早早压在低位反复震荡，v2 的原始曲线抖动幅度最大

这张图把前面两节的”毛病”变成了可以看见的东西。**细线是每次迭代的原始奖励**：v1 和 v2 的细线一直毛刺丛生，这就是 4.7 节说的回报噪声——学习信号被淹没在抖动里；v3 的细线几乎贴着粗线走，说明每次更新都稳。**粗线是趋势**：v3 全程稳稳压在另外两条之上，绝大部分区间稳步爬升；v1 和 v2 大约 700 次迭代后就双双卡在 0 附近再也上不去，v2 甚至比 v1 更低更抖——这正是 5.6 节说的”更新步子没人管”。

训练时记下的各项损失，则从另一个角度说明了三版到底各自在优化什么：

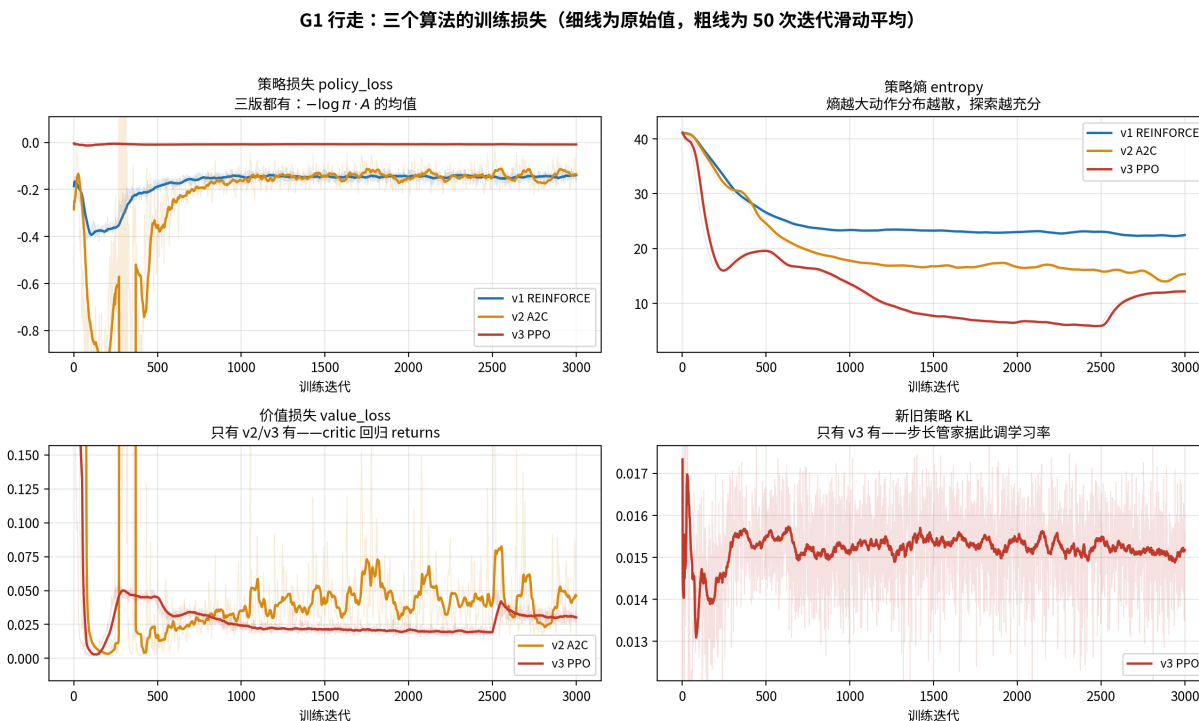


图 15: G1 行走三算法的训练损失（细线为每次迭代原始值，粗线为 50 次迭代滑动平均）：左上策略损失三版都有；右上策略熵反映探索强度；左下价值损失只有 v2/v3 有；右下新旧策略 KL 只有 v3 记录

四个面板正好对应三版的差异。**策略损失**三版都有，但只有它是不够的——它的绝对值不能横向比较，因为每版的优势估计口径不同。**策略熵**最能说明问题：三条线都从 41 附近往下走（策略逐渐变确定），但走的节奏差很多——v1 降得最慢、末段仍停在 22 以上，说明它的动作分布一直很散、迟迟收不拢；v3 一路压到 6 附近（三者中最低），动作最果断。**价值损失**只有 v2 和 v3 有，因为 v1 根本没训 critic；两条线都稳定收敛，说明评分员确实学到了东西——这也印证了 5.5 节的判断：A2C 卡住不是因为 critic 没学好。**新旧策略 KL** 只有 v3 记录，全程稳定在 0.015 附近——这正是步长管家在起作用：KL 偏大就压学习率、偏小就放开，把每次更新的幅度锁在一条窄带里。

v3 把两个指标同时拿下：评测奖励最高（0.097），**摔倒率归零**。它对 5.5 节那个反常结果的回收很有说服力：A2C 那版的优势估计并没有问题，卡脖子的是步长——补上 clip 之后，同样的优势估计立刻兑现成了最高分。而且 v3 其实比 v2 训得更激进：同一批数据训 5 个 epoch、每个 epoch 切 4 份 minibatch，一共 20 个优化 step（单个样本约被用 5 次，不是 20 次——每个 epoch 里每个样本只出现在一份 minibatch 里），而 v2 只过一遍。更激进却更稳，是因为 clip 把每次更新的概率比压在了一个小区间里。**护栏不是让车变慢，是让车敢开快。**

备注：3000 迭代的课堂预算下，三者都还停留在“稳住、小幅行走”而非利落快走（官方配方约需 3 万迭代才走得漂亮）。本实验的目的是对照算法差异，不是刷行走质量；把 `max_iterations` 调大即可提升绝对效果，算法本身不用动。

6.6 实验加码：动作跟随，任务越难差距越大

行走任务上三版好歹都能站住，差距还不算触目。把同一套代码搬到一个**难得多**的任务上试试：让 G1 **逐帧贴住一段真实武术动作**——参考动作由真人视频重定向到 G1 得到，用的正是 4.4 节认识的那条管线：GVHMR 从视频恢复人体动作，GMR 把它重定向成 G1 的参考轨迹。它比走路难在目标上：不再是“别摔就行”，而是每一帧都要匹配参考姿态。

这正是 BeyondMimic 做的事情——同一套 PPO 配方，训练真实宇树 G1 做出敏捷的踢腿、快步、翻转：

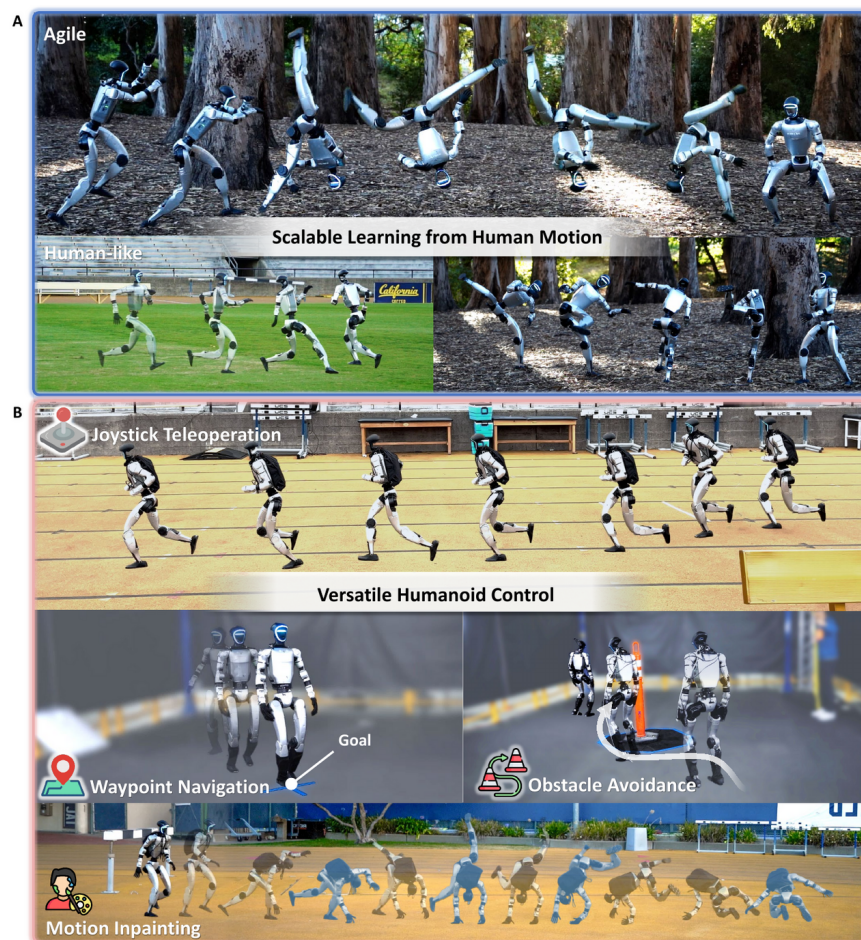


图 16: BeyondMimic 在真实宇树 G1 上的效果：上半部分是从人体动作中学到的敏捷与拟人动作，下半部分是同一套策略支持的手柄遥操、路点导航、避障等多种全身控制

备注：BeyondMimic 是**动作追踪**——让机器人稳稳执行给定的动捕轨迹，而不是凭空“涌现”出跑酷。这一点要说清楚，免得对强化学习的能力产生误解：难点在控制，不在动作创意。

仿真里跑同样的三档，但要先把口径讲清楚：这一组不是公平的算法对照。v3 直接复用了原版 PPO 训到 10000 迭代的权重，而 v1/v2 只训了 3000 迭代——训练预算差了三倍多，算法差异和训练时长差异混在了一起。所以下表应当读作“各版本最终可用 checkpoint 的展示”，不能用来支撑“算法 A 强

于算法 B”:

版本	算法 (训练预算)	评测平均奖励	摔倒率
v1	REINFORCE (3000 迭代)	0.052	2.6%
v2	A2C (3000 迭代)	0.046	1.2%
v3	PPO 原版 (10000 迭代)	0.070	0.0%

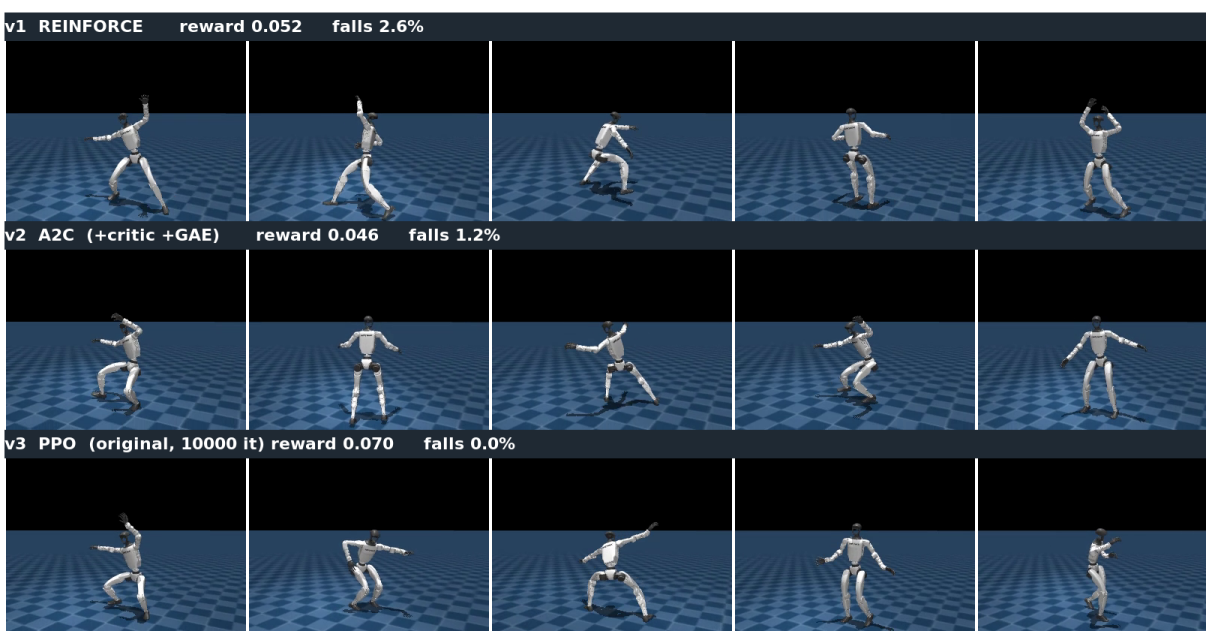


图 17: G1 动作跟随三算法 rollout 对照，每行从左到右是同一段时间内的关键帧，从上到下依次为 v1 REINFORCE、v2 A2C、v3 PPO: v3 做出干净协调的武术姿态（宽马步、抬臂出击）且从不摔倒，v1/v2 明显跟不住参考、东倒西歪

图里能直接看到的是：底行的 PPO 打出干净、协调的武术动作，宽马步、抬臂出击都稳稳当当；上面两行东倒西歪，跟不住参考。但”任务越难差距越大”这个结论在这组数字上还立不住：训练预算不对齐，算法差异和训练时长混在一起，要分开就得统一环境步数 / 墙钟 / 优化 step 并跑多个随机种子。所以这里只取一句更保守的观察：在各自这个预算下，完整 PPO 的最终 checkpoint 在硬任务上明显更能跟住参考。真正预算对齐的对照是行走任务那组（6.5 节，三者同为 3000 迭代）。

备注（本讲实验结论的效力边界）：6.5 节与本节的数字都来自**单次训练、单次评测**，没有多随机种子、没有置信区间，评测回合数也有限；rollout 截图只是定性展示，不能替代统计。要把这类对照写进正式结论，标准做法是：报告训练曲线的均值 $\pm$ 标准差（至少 3~5 个种子）、评测至少 100 回合并给出成功率区间、公开评测脚本与 checkpoint 的选取规则。本讲的目的是让你看清三个算法**机制**上的差别，教学演示的数字请按”本次运行观察到”来读。

6.7 本节小结

PPO 在 A2C 之上补的是”步长安全”：用概率比值 r_t 度量策略变化，clip 让”走太远”无利可图，软性地把更新压在旧策略附近；有 clip 抑制过大更新才敢多轮复用数据，再加 KL 自适应学习率自动收放步长。在同预算的行走任务对照里（6.5 节），本次运行中 v3 同时拿下最高奖励与零摔倒，回收了 v2 的反常；动作跟随任务上 PPO 的最终 checkpoint 明显更能跟住参考（注意那一组训练预算不对齐，见 6.6 节）。

7 本讲小结

7.1 一条演进链：每一步为什么发生

回看整条路，每个算法都是被前一个的毛病逼出来的：

版本	算法	新增机制	解决什么毛病	对应章节
v1	REINFORCE	策略梯度 + reward-to-go	——（起点）	第 4 节
v2	A2C	critic 基线 + GAE 优势	v1 回报噪声大（方差）	第 5 节
v3	PPO	clip + 多轮复用 + KL 自适应	v2 更新步长没人管	第 6 节

一轮完整 PPO 的数据流也顺一遍：当前策略在 4096 个并行环境里采 24 步 → critic + GAE 把每步的优势估出来 → 切成 4 份 minibatch 反复训 5 轮，policy loss 走 clip、value loss 回归回报 → 按 KL 调学习率 → 丢掉数据，用新策略再采。本讲的关键超参数一览：

超参数	取值	出处
折扣因子 γ	0.99	2.3 节
GAE 系数 λ	0.95	5.3 节
clip 范围 ϵ	0.2	6.2 节
复用轮数 $\times$ minibatch 数	5×4	6.3 节
目标 KL（学习率自适应）	0.01	6.3 节
熵奖励系数	0.01	4.5 节
并行环境 $\times$ 每轮步数 $\times$ 迭代	$4096 \times 24 \times 3000$	1.4 节

概念多，收一张速查表，每个词都能找到它第一次出现的地方：

名词	一句话	首次出现
奖励 / 回报	每步的分 / 一整段的折扣总分	2.3 节
基于价值 / 基于策略	学” 值多少分” / 直接学” 怎么做”	3.2 节
策略梯度方法	基于策略一族中沿梯度上升的子类	3.2 节
actor-critic	策略网络 + 价值网络的混合架构，横跨基于价值与基于策略两类	3.3 节
on-policy / off-policy	学的策略和采数据的策略是否同一个	3.4 节
reward-to-go	只算这一步之后的回报	4.2 节
基线 / 优势	参照物 / 比这个状态的平均水平好多少 ($A = Q - V$)	5.1 节 / 5.2 节
bootstrap (自举)	用价值估计替代部分实际回报；REINFORCE 与 actor-critic 的分界	5.3 节
GAE	多步时序差分残差的 λ 加权，估优势	5.3 节
信任域 / clip	更新只许在旧策略附近 / 用裁剪实现它	6.1 节 / 6.2 节

7.2 回到分类地图：GRPO 与 off-policy 在等下两讲

用第 3 节的三个划分给本讲定位：我们全程走在 model-free、基于策略（策略梯度）、on-policy 这一格里，把这一格内部从最朴素做到了工业级。地图上还有两步棋，正好是下两讲：

往前一站是 GRPO（第 15 讲）。场景换成大模型和 VLA 的后训练：奖励从”走得稳不稳”变成可验证的对错，“动作”从关节目标量变成文本 token。这个场景里 critic 变麻烦了——在大模型 RLHF 里再挂一个价值网络会明显推高显存与计算，具体增加多少取决于它是否与策略共享骨干（常见做法是共享 backbone、只加一个 value head，也可以用更小的网络或冻结部分模块），以及并行 / offload 怎么实现，并不是简单的”翻倍”；但这个场景也有个本讲没有的便利：同一道题可以让模型采样一组回答，组内的平均分天然就是”这个状态的平均水平”。于是 GRPO 做了个漂亮的取舍：**保留 PPO 的 clip，请走 critic**，用组内相对优势直接当权重。critic 在第 5 节被我们请进来、到 GRPO 又被请走——不是倒退，是换了场景之后，对”参照物从哪来”这个老问题的更省答案。你会发现本讲建立的优势、信任域、clip 这套直觉，原样迁移过去。

另一格是 off-policy 的价值线（第 16 讲）。真机上一条数据都金贵，on-policy”用完即弃”养不起，必须换到 off-policy、把经验池榨干——也就是 3.5 节表里 Q-learning、DQN 到 SAC 那一列。第 16 讲的 HIL-SERL 就建在 SAC 上，还要把人拉进训练环里。到那时，第 3 节这张地图上的每一格就都被我们亲手踩过了。

一句话总结：强化学习用奖励代替标准答案，让机器人在试错中学会示教教不了的事；REINFORCE 证明”好动作加概率”可行，A2C 用 critic 把信号洗干净，PPO 用 clip 管住步子——G1 从晃晃悠悠站住，到零摔倒行走，再到打出一套干净的武术动作，就是这条演进链一步一个脚印的直观证据。

参考资料

- [1] ACHIAM J. Spinning up in deep reinforcement learning[EB/OL]. (2018)[2026-06-26]. <https://spinningup.openai.com>.
- [2] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms[EB/OL]. (2017-07-20)[2026-06-26]. <https://arxiv.org/abs/1707.06347>.
- [3] SCHULMAN J, MORITZ P, LEVINE S, et al. High-dimensional continuous control using generalized advantage estimation[EB/OL]. (2015-06-08)[2026-06-26]. <https://arxiv.org/abs/1506.02438>.
- [4] LIAO Q, TRUONG T E, HUANG X, et al. BeyondMimic: from motion tracking to versatile humanoid control via guided diffusion[EB/OL]. (2025-08-11)[2026-06-26]. <https://arxiv.org/abs/2508.08241>.
- [5] SUTTON R S, BARTO A G. Reinforcement learning: an introduction[M]. 2nd ed. Cambridge: MIT Press, 2018.
- [6] SIMONINI T, SANSEVIERO O. The Hugging Face deep reinforcement learning course[EB/OL]. (2023)[2026-06-26]. <https://huggingface.co/learn/deep-rl-course>.
- [7] ZAKKA K, YI B, LIAO Q, et al. mujlab: a lightweight framework for GPU-accelerated robot learning[EB/OL]. (2025)[2026-07-02]. <https://github.com/mujocolab/mujlab>.
- [8] SHEN Z, PI H, XIA Y, et al. World-grounded human motion recovery via gravity-view coordinates[EB/OL]. (2024-09-10)[2026-07-02]. <https://arxiv.org/abs/2409.06662>.
- [9] ARAUJO J P, ZE Y, XU P, et al. Retargeting matters: general motion retargeting for humanoid motion tracking[EB/OL]. (2025-10-02)[2026-07-02]. <https://arxiv.org/abs/2510.02252>.
- [10] ZE Y. GMR: general motion retargeting[EB/OL]. (2025-08-04)[2026-07-02]. <https://github.com/YanjieZe/GMR>.